Project #4: Mallocator

20231508 강유영

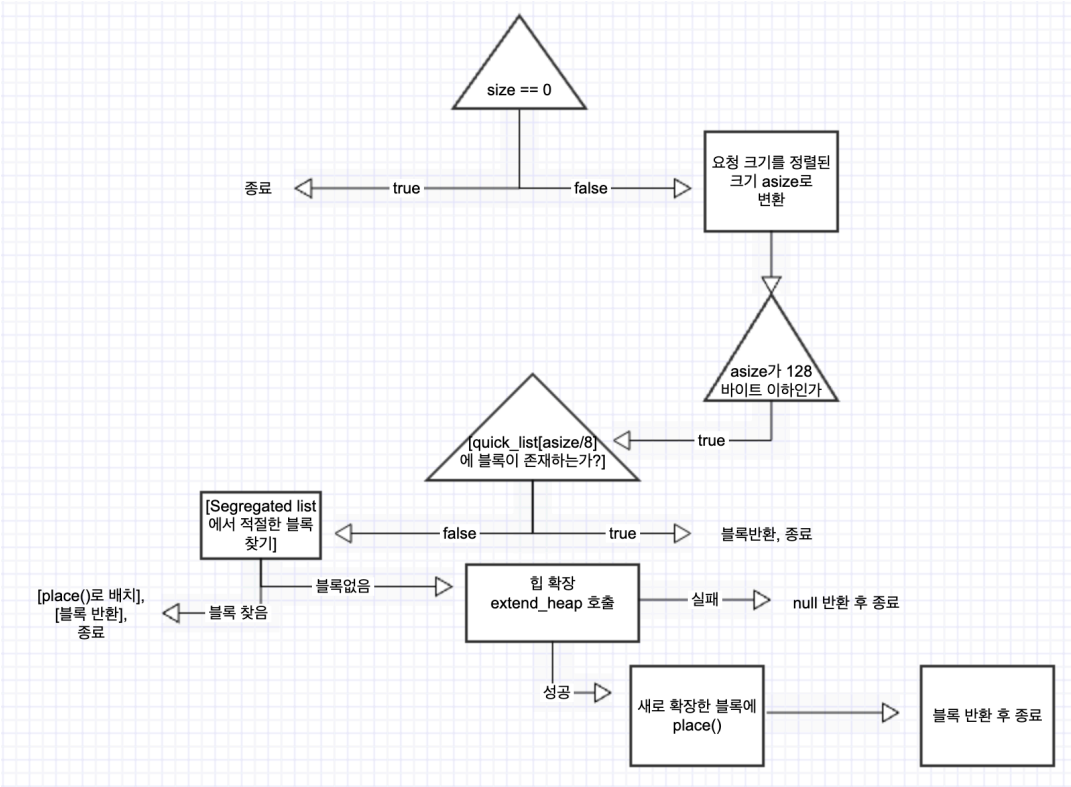
1. 전체 구조 설계

본 할당기는 segregated free list와 quick list를 혼합하여 메모리 요청 크기에 따라 다르게 처리하는 구조로 되어 있다. 전체 구조는 다음과 같다.

- 메모리 힙의 시작에는 prologue block(헤더 8바이트), epilogue block이 존재한다.
- 요청된 크기가 작을 경우 (128바이트 이하) quick list를 먼저 탐색하여 즉시 재사용 가능한 블록을 반환한다.
- 요청된 크기가 큰 경우에는 segregated free list 중 적절한 클래스를 선택하여 해당 리스트를 탐색한다.
- 적절한 블록이 없을 경우 mem_sbrk을 통해 힙을 확장한 후 새로운 블록을 생성하여 반환한다.
- free된 블록은 인접 블록이 free 상태일 경우 coalescing을 통해 병합된다.
- realloc 시 기존 블록의 다음 블록이 free이면서 크기가 충분할 경우 in-place 확장이 이루어지며, 그렇지 않으면 새 블록을 할당하고 데이터를 복사한 후 기존 블록을 해제한다.

2. Allocater 설계

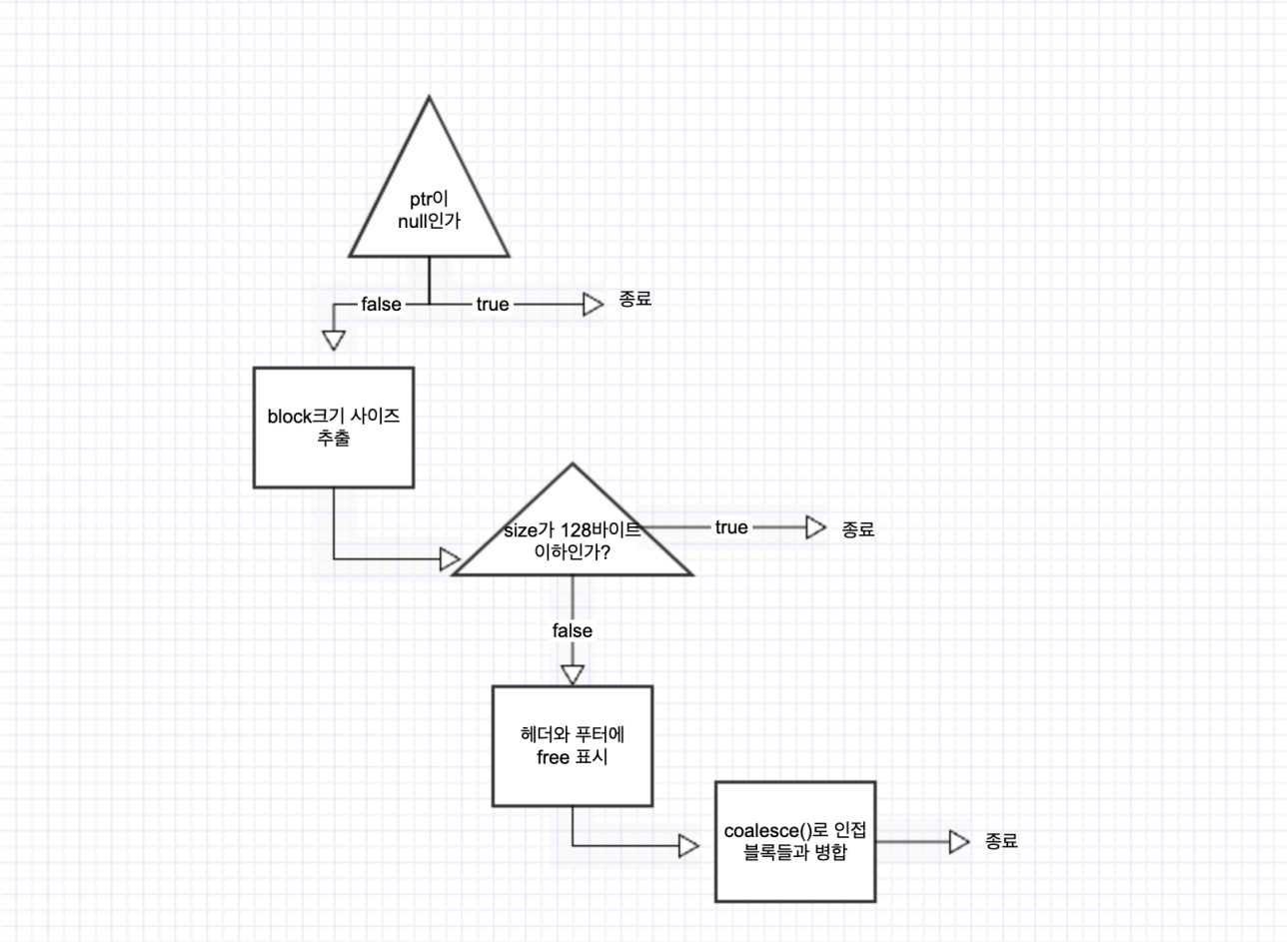
2-1. void *mm_malloc(size_t size)



요청된 크기만큼 메모리를 할당한다.

- 요청 크기를 정렬(adjust_size)하여 실제 필요한 블록 크기(asize)를 구함.
- asize가 128바이트 이하이면 Quick List에서 할당 가능한 블록을 가져옴.
- 아니면 Segregated Free List에서 적절한 블록을 find_fit()으로 검색.
- 적절한 블록이 없을 경우 extend_heap()으로 힙을 확장한 뒤 place()를 통해 블록을 배치.

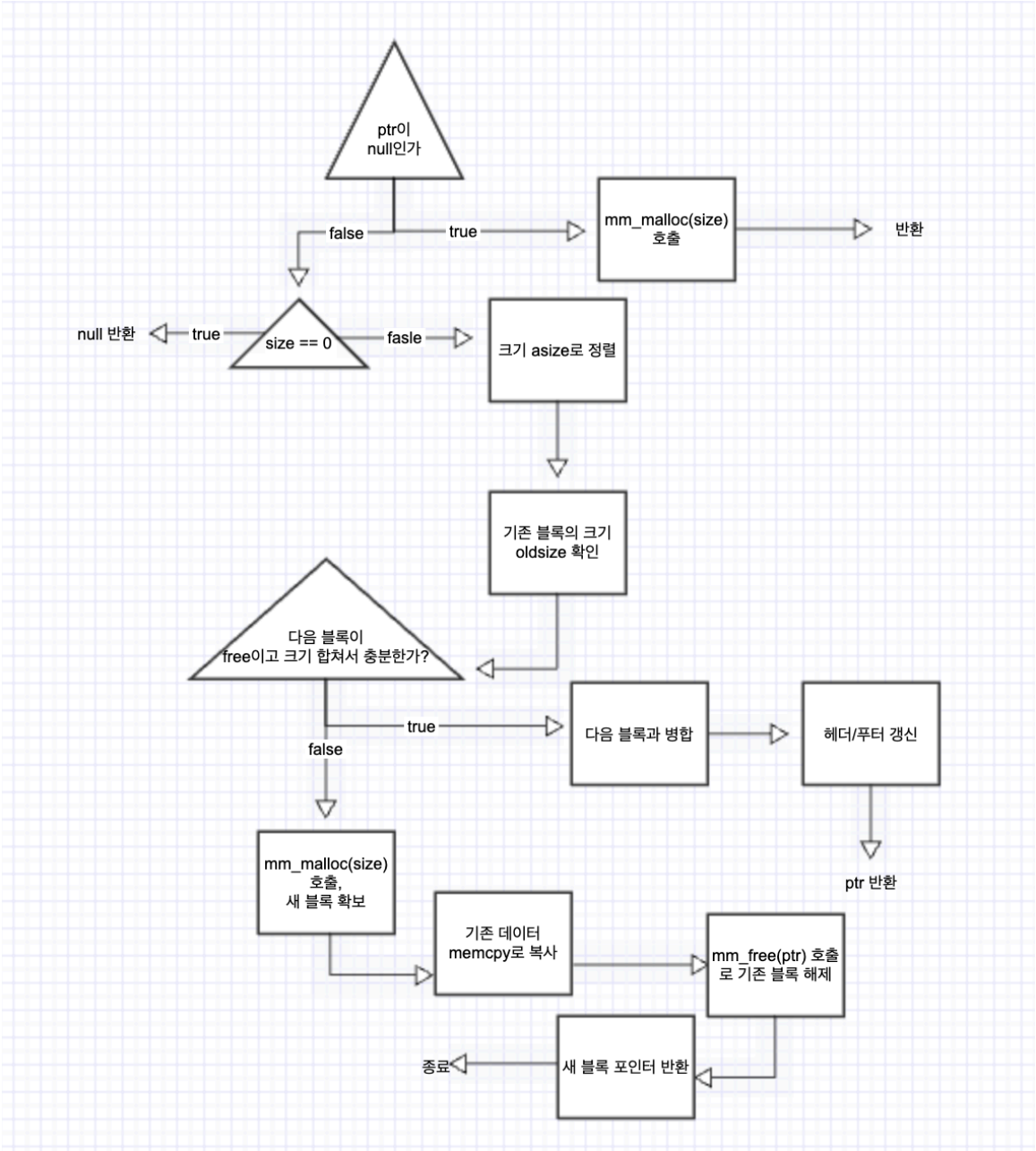
2-2. void mm_free(void *ptr)



사용이 끝난 메모리를 해제하고 다시 할당 가능한 상태로 만든다.

- NULL 포인터는 무시.
- 블록 크기가 128바이트 이하이면 Quick List에 넣어 빠른 재사용이 가능하게 함.
- 그렇지 않으면 coalesce()를 통해 인접한 free 블록과 병합 후, Segregated List에 넣음.

2-3. void *mm_realloc(void *ptr, size_t size)



이미 할당된 메모리의 크기를 size만큼 재할당한다.

- ptr == NULL이면 mm_malloc()을 호출.
- size == 0이면 mm_free()를 호출.
- 기존 블록의 다음 블록이 비어 있고 크기가 충분한 경우에는 in-place 확장 수행.
- 그렇지 않으면 새 블록을 mm_malloc()로 받아서 데이터를 복사(memcpy) 후 기존 블록을 mm_free().

3. 전역 변수

- team: 과제 요구사항에 따라 팀원 정보를 저장하는 구조체 변수이다.
- seg_list[16]: segregated free list의 각 크기 클래스의 헤더를 저장하는 static 배열이다. 블록의 크기에 따라 적절한 리스트에 연결된다.
- quick_lists[16]: 128바이트 이하의 소형 블록을 저장하는 quick list의 헤더를 저장하는 static 배열이다.
- rover: next-fit 검색을 위한 static 포인터로, 마지막 탐색 위치를 저장한다.

위 전역 변수들은 할당기 전체에서 공유되며, 구조의 간결성과 성능 향상을 위해 static으로 선언되어 있다. 전역 변수는 최소화되었으며, 전역 배열을 제외한 다른 전역 변수는 존재하지 않는다.

4. 구조체

4-1. team_t

본 할당기 구현에서는 직접 정의한 구조체는 사용하지 않았으며, 과제 명세에서 제공된 구조체인 team_t만 사용하였다. 메모리 블록의 연결 및 관리는 구조체를 정의하지 않고 포인터 연산을 통해 직접 구현하였다.

이 구조체는 제출 시 자동 채점 프로그램에서 식별 목적으로 사용되며, 메모리 할당기의 동작과는 직접적인 연관이 없다.

4-2. 메모리 블록 구조

블록 관리는 구조체 대신 포인터 연산을 사용하여 구현하였다. 각 블록은 다음과 같은 형식으로 메모리에 배치된다.

- allocated block
  [header][payload]
  header에는 블록의 크기와 할당 여부가 포함되며, payload는 사용자 데이터가 저장되는 영역이다.
- free block
  [header][prev][next][…padding…][footer]
  prev와 next 포인터는 free list에서 이중 연결 리스트를 구현하기 위해 사용된다. 이들은 포인터 연산을 통해 접근하며, 구조체를 정의하지 않고 메모리 상에서 직접 조작한다.

예를 들어, free block의 next 포인터는 다음과 같이 접근한다.

```
*(void **)bp          // next pointer

*(void **)((char *)bp + WSIZE) // prev pointer
```

이와 같은 방식으로 구조체 없이도 블록 간 연결 및 삽입/삭제 연산을 수행할 수 있다.

결론적으로, 본 구현에서는 구조체를 최소화하고 메모리 공간을 효율적으로 활용하기 위해 포인터 연산을 사용하였다. 과제의 전역 변수 및 구조체 사용 최소화 지침을 충실히 따르며, 오직 team 정보를 위한 team_t 구조체만 사용하였다.

5. 서브루틴

- mm_init(): 전체 힙을 초기화하고 prologue 및 epilogue 블록을 설정한 후, 초기 free 블록을 할당하기 위해 extend_heap을 호출한다.
- mm_malloc(size_t size): 요청된 크기를 정렬 후 quick list나 segregated list에서 적절한 free 블록을 검색하여 배치한다. 적절한 블록이 없을 경우 힙을 확장한다.
- mm_free(void *ptr): 주어진 포인터가 가리키는 블록을 해제하고, 크기에 따라 quick list 또는 seg list에 삽입한다. seg list에 삽입하기 전에 coalescing을 통해 병합이 수행된다.
- mm_realloc(void *ptr, size_t size): 기존 블록의 크기를 고려하여 in-place 확장을 시도하고, 불가능한 경우 새 블록을 할당한 후 복사하고 기존 블록을 해제한다.
- coalesce(void *bp): 인접 블록들과의 병합을 처리한다. 앞, 뒤 블록이 free인지 여부에 따라 총 4가지 경우로 나뉘어 처리된다.
- find_fit(size_t asize): asize 이상 크기를 가진 블록을 next-fit 방식으로 찾는다. rover 포인터를 이용해 순환 탐색을 수행한다.
- place(void *bp, size_t asize): 블록을 할당하고, 남는 공간이 있다면 분할하여 나머지를 free 블록으로 유지한다.
- extend_heap(size_t words): mem_sbrk을 통해 힙을 확장하고 새로운 free 블록을 생성한다.
- insert_block(void *bp), remove_block(void *bp): segregated list에 블록을 삽입하거나 삭제할 때 사용하는 함수로, 이중 연결 리스트를 조작한다.
- adjust_size(size_t size): 요청 크기를 더블 워드 정렬에 맞게 조정하는 함수이다.
- class_index(size_t size): 블록 크기에 따라 적절한 segregated list 인덱스를 계산한다.

이상의 서브루틴은 전체 allocator의 기능을 modular하게 나누어 설계 및 구현되었으며, 각 함수는 독립적으로 동작하면서 전체 구조와 유기적으로 연결된다.